

DOCUMENTAȚIE TEHNICĂ

TasteTest

Platformă de descoperire, evaluare și recenzare a restaurantelor, construită ca sistem de microservicii Spring Boot cu frontend Angular și asistent AI.

Spring Boot 3.4.5

Spring Cloud 2024.0.1

Java 21

Angular 21

PostgreSQL · Redis · MongoDB

Docker Compose

Spring AI

Proiect: Aplicații Web și Baze de Date (AWDB) · **Arhitectură:** Microservicii

Autor: Mihai Popescu · **Versiune document:** 1.0 · **Data:** 20 iunie 2026

Cuprins

1. Prezentare generală a proiectului	3
Scop, context academic, capabilități funcționale, glosar.	
2. Stack tehnologic complet	6
Backend, cloud, frontend, baze de date, observabilitate, AI, DevOps.	
3. Arhitectura de ansamblu	10
Harta microserviciilor, comunicare, diagramă de sistem.	
4. Componente de infrastructură	13
Discovery Server, Config Server, API Gateway.	
5. Microservicii de business	16
auth, user, restaurant, notification, agent, monolit legacy.	
6. Modelul de date	20
Schema relațională, migrări Flyway, ERD, NoSQL.	
7. Securitate și autorizare	24
JWT HS512, roluri, fluxul token-ului, filtre.	
8. Decizii arhitecturale (ADR)	27
Saga, caching, NoSQL, config central, gateway, MFE, AI.	
9. Fluxuri de utilizare	31
Înregistrare, login, adăugare restaurant, recenzii, asistent, admin.	
10. Frontend (Angular)	34
Shell-ul principal, micro-frontend admin, structură, rute.	
11. Referință API	37
Toate endpoint-urile, pe serviciu, cu roluri.	
12. Observabilitate și monitorizare	40
Actuator, Prometheus, Grafana, Zipkin.	
13. CI/CD	41
GitHub Actions, build, teste, imagini Docker, staging.	
14. Rulare, profiluri și deployment	42
Profiluri Spring, Docker Compose, load balancing.	
15. Troubleshooting și îmbunătățiri viitoare	44
Probleme frecvente, limitări cunoscute, roadmap.	

1. Prezentare generală a proiectului

TasteTest este o platformă web prin care utilizatorii descoperă restaurante, le evaluează cu note și recenzii, iar proprietarii de restaurante își administrează localurile, meniurile și răspund la recenzii. Aplicația a fost construită inițial ca monolit Spring Boot și apoi descompusă într-un set de microservicii independente, conectate printr-un service registry și expuse printr-un API Gateway.

1.1 Scop și context

Proiectul a fost dezvoltat pentru disciplina **Aplicații Web și Baze de Date (AWDB)**. Pe lângă funcționalitatea de bază (CRUD pe restaurante, meniuri, recenzii, utilizatori), proiectul acoperă o serie de cerințe avansate din „Partea a II-a” a temei: arhitectură de microservicii, securitate cu JWT, un design pattern distribuit (Saga), integrare NoSQL și caching, micro-frontends, CI/CD, observabilitate și un agent AI integrat în aplicație.

Documentul de față descrie sistemul în întregime: tehnologiile folosite, arhitectura, fiecare serviciu în parte, modelul de date, securitatea, deciziile arhitecturale cu motivația din spatele lor, fluxurile de utilizare end-to-end, frontend-ul, referința completă de API, observabilitatea, pipeline-ul CI/CD și procedurile de rulare.

1.2 Capabilități funcționale

La nivel de produs, platforma oferă următoarele capabilități, grupate pe domenii:

Domeniu	Capabilități
Autentificare & autorizare	Înregistrare cont, autentificare, emitere și validare token JWT. Trei roluri: ADMIN , USER , RESTAURANT_OWNER . Autorizare la nivel de endpoint, pe baza rolurilor.
Restaurante	Un RESTAURANT_OWNER (sau ADMIN) adaugă, editează și șterge restaurante. Fiecare restaurant are denumire unică, adresă, telefon, site și program.
Categorii	Restaurantele pot avea una sau mai multe categorii culinare (ex.: românesc, italian, fast food), administrate independent.
Meniuri & articole	Fiecare restaurant are unul sau mai multe meniuri; fiecare meniu are articole cu denumire, preț și descriere.
Recenzii & răspunsuri	Utilizatorii lasă recenzii (notă 1–5 și comentariu). Reprezentantul restaurantului poate răspunde la o recenzie.
Feedback	Utilizatorii trimit feedback general despre experiența din aplicație (anonim sau autentificat).
Căutare & filtrare	Căutare după nume/categorie/locație; filtrare după rating și specific culinar; listă „top-rated”; paginare.
Administrare	Administratorii gestionează utilizatorii, rolurile, restaurantele, categoriile și feedback-ul, dintr-un panou separat (micro-frontend).
Asistent AI (TasteBot)	Chat in-app care răspunde la întrebări despre catalog folosind „tool calling” peste serviciile reale, fără a inventa date.

1.3 Glosar de termeni

Termen	Semnificație în contextul TasteTest
Microserviciu	Aplicație Spring Boot independentă, cu propriul <code>pom.xml</code> , <code>Dockerfile</code> și ciclu de viață, înregistrată în Eureka.
Eureka / Discovery	Registry de servicii. Fiecare serviciu se înregistrează la pornire și descoperă celelalte servicii după nume logic.
API Gateway	Punctul unic de intrare public. Rutează cererile către microserviciul potrivit pe baza căii din URL.
Config Server	Serviciu care servește configurația centralizată din folderul <code>config-repo/</code> către toate celelalte servicii.
JWT	JSON Web Token semnat HS512, emis de auth-service și acceptat de toate celelalte servicii.
Saga	Pattern pentru tranzacții distribuite: pași locali + acțiuni de compensare în caz de eșec.
MFE	Micro-frontend — aplicație Angular independentă (panoul de admin), încărcată în shell printr-un iframe.
Tool calling	Mecanismul prin care modelul AI apelează funcții (unelte) ce împachetează endpoint-uri REST reale.

2. Stack tehnologic complet

Proiectul combină un backend Java/Spring în arhitectură de microservicii, un frontend Angular dublat de un micro-frontend, persistență poliglotă (relațional + două baze NoSQL) și un stack complet de observabilitate și CI/CD.

2.1 Backend & platformă cloud

Tehnologie	Versiune	Rol în proiect
Java	21 (LTS)	Limbajul de bază pentru toate serviciile backend.
Spring Boot	3.4.5	Framework-ul de bază (web, security, data, actuator). Parent POM pentru toate modulele.
Spring Cloud	2024.0.1	BOM pentru componentele distribuite (Config, Eureka, Gateway, LoadBalancer).
Spring Cloud Netflix Eureka	—	Service discovery: server (discovery-server) + clienți.
Spring Cloud Config	—	Configurare centralizată, citită din <code>config-repo/</code> .
Spring Cloud Gateway	—	API Gateway reactiv (pe WebFlux/Netty), rutare pe bază de path.
Spring Cloud LoadBalancer	—	Balansare client-side a apelurilor inter-servicii (<code>lb://</code>).
Spring Security	—	Autentificare/autorizare, filtre JWT, <code>@PreAuthorize</code> pe metode.
Spring Data JPA + Hibernate	—	Maparea entităților relaționale (PostgreSQL/H2).
Spring Data Redis	—	Abstracția pentru cache-ul Redis din <code>restaurant-service</code> .
Spring Data MongoDB	—	Persistența documentelor de notificare din <code>notification-service</code> .
Spring AI	1.0.0	Integrarea modelului LLM și mecanismul de tool calling (<code>agent-service</code>).
JWT (io.jsonwebtoken)	—	Generarea și validarea token-urilor JWT (HS512).
Flyway	—	Migrări versionate de schemă pentru PostgreSQL.
Springdoc OpenAPI / Swagger UI	—	Documentație API interactivă pe servicii.
Maven (Wrapper)	—	Build multi-modul (reactor) prin <code>./mvnw</code> .

2.2 Frontend

Tehnologie	Versiune	Rol în proiect
Angular	21.2	Shell-ul principal (frontend/) și micro-frontend-ul de admin (mfe-admin/).
TypeScript	5.9	Limbajul aplicațiilor Angular.
RxJS	7.8	Programare reactivă pentru apelurile HTTP și starea componentelor.
Angular Router + Standalone APIs	—	Rutare, lazy bootstrap, componente standalone (fără NgModule).
Nginx	1.27-alpine	Servește build-ul de producție și face proxy /api către gateway.
Vite (Angular build)	—	Toolchain-ul de build/dev-server al Angular 21.

2.3 Persistență (poliglotă)

Sistem	Versiune	Folosit de	Rol
PostgreSQL	16-alpine	auth, user, restaurant, backend-original	Baza de date relațională principală (schema proiect).
H2	in-memory	profilul test	BD efemeră pentru teste și demo rapid, fără Docker.
Redis	7-alpine	restaurant-service	Caching layer (TTL) pentru citiri costisitoare.
MongoDB	7	notification-service	Bază NoSQL pentru istoricul notificărilor (documente).

2.4 Observabilitate, AI & DevOps

Tehnologie	Versiune	Rol în proiect
Spring Boot Actuator	—	Endpoint-uri de health, metrics, info, refresh, prometheus.
Micrometer + Prometheus	v2.54.1	Expunerea și colectarea metricilor (CPU, memorie, request rate).
Grafana	11.4.0	Dashboard de observabilitate, provisionat automat din Prometheus.
Zipkin + Micrometer Tracing (Brave)	2.24.3	Tracing distribuit între servicii.
Anthropic Claude Haiku 4.5 / LLM local	—	Modelul folosit de TasteBot (configurabil: Anthropic sau LM Studio compatibil OpenAI).
Docker & Docker Compose	—	Împachetare și orchestrare locală a întregului stack (16+ containere).
Podman	—	Alternativă la Docker, suportată prin <code>run-podman.sh</code> .
GitHub Actions	—	Pipeline CI/CD: build, teste, publicare imagini în GHCR.
GitHub Container Registry (GHCR)	—	Registry pentru imaginile Docker (<code>:staging</code> , <code>:sha-<commit></code>).

De ce această combinație

Stack-ul este deliberat „mainstream enterprise”: ecosistemul Spring Cloud oferă out-of-the-box toate piesele unei arhitecturi de microservicii (discovery, config, gateway, load balancing), iar Docker Compose le leagă local într-un mediu reproductibil. Persistența poliglotă (PostgreSQL + Redis + MongoDB) demonstrează că fiecare tip de date este pus în depozitul potrivit, nu totul într-o singură bază relațională.

3. Arhitectura de ansamblu

Aplicația a fost desfăcută dintr-un monolit într-un set de microservicii independente, fiecare cu propria responsabilitate. Serviciile se înregistrează în Eureka, își iau configurația din Config Server și sunt expuse public printr-un singur API Gateway. Comunicarea internă se face prin REST balansat client-side peste numele logice din Eureka.

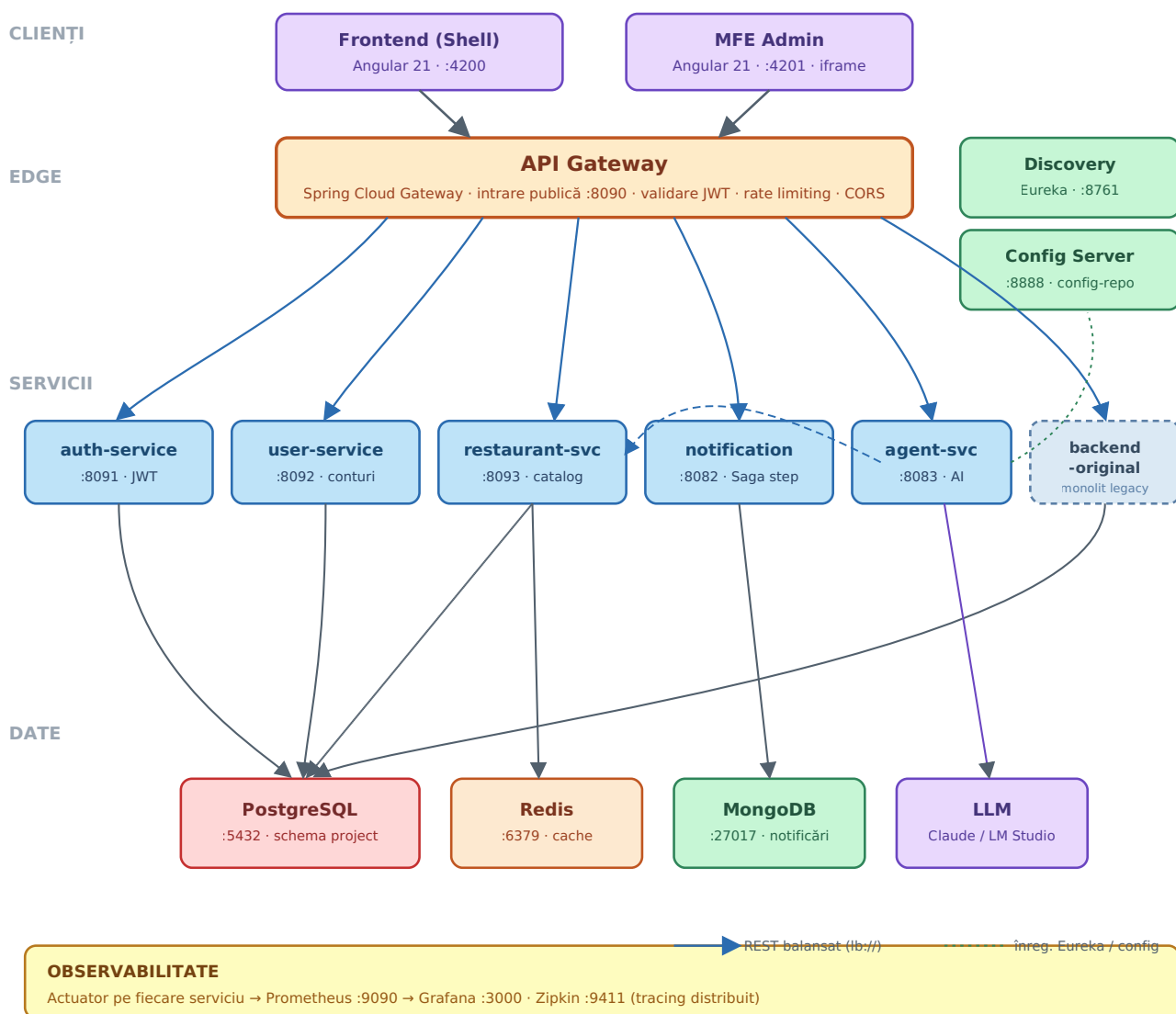


Figura 3.1 — Arhitectura de ansamblu: clienți, gateway, servicii de platformă și de business, depozite de date și observabilitate.

3.1 Principii de proiectare

Arhitectura respectă câteva principii consacrate pentru sisteme distribuite:

- **Un singur punct de intrare.** Tot traficul extern trece prin API Gateway (`http://localhost:8090`), care rutează pe baza căii. Clienții nu cunosc porturile interne ale serviciilor.
- **Descoperire dinamică, nu adrese hardcodate.** Apelurile inter-servicii sunt scrise ca `http://<nume-eureka>/...` și rezolvate la runtime de Spring Cloud LoadBalancer la o instanță concretă.
- **Configurare externalizată.** Secretele și parametrii (JWT, datasource, mail) stau în `config-repo/`, nu în cod, și pot fi reîncărcate la cald cu `POST /actuator/refresh`.
- **Securitate uniformă.** Un singur secret HS512 partajat permite oricărui serviciu să valideze token-ul emis de auth-service, fără apeluri suplimentare.
- **Persistență poliglotă.** Datele relaționale stau în PostgreSQL, cache-ul în Redis, istoricul de notificări în MongoDB — fiecare în depozitul potrivit.
- **Observabilitate încorporată.** Fiecare serviciu expune Actuator; Prometheus colectează metrice, Grafana le vizualizează, Zipkin urmărește trace-urile.

3.2 Inventarul serviciilor și porturilor

Serviciu	Director	Nume Eureka	Port intern	Port host	Rol
Discovery Server	discovery-server/	tastetest-discovery-server	8761	8761	Registry Eureka
Config Server	config-server/	tastetest-config-server	8888	8888	Configurare centralizată
API Gateway	api-gateway/	tastetest-api-gateway	80	8090	Intrare publică, rutare
Auth Service	auth-service/	tastetest-auth	8091	8091	Autentificare + emitere JWT
User Service	user-service/	tastetest-user	8092	8092	Conturi, roluri, recenzii
Restaurant Service	restaurant-service/	tastetest-restaurant	8093	8093	Restaurante, meniuri, rating
Notification Service	notification-service/	tastetest-notification-service	8082	8082	Notificări (pas Saga)
Agent Service	agent-service/	tastetest-agent	8083	8083	Asistent AI (TasteBot)
Backend original	backend-original/	tastetest-awdb	8090	— (intern)	Monolit legacy
Frontend (shell)	frontend/	—	80	4200	UI principal
MFE Admin	mfe-admin/	—	80	4201	Micro-frontend admin
PostgreSQL	— (image)	—	5432	5432	BD relațională
Redis	— (image)	—	6379	6379	Cache
MongoDB	— (image)	—	27017	27017	NoSQL notificări
Prometheus / Grafana / Zipkin	— (imagini)	—	—	9090 / 3000 / 9411	Observabilitate

Comunicarea inter-servicii

Exemplu concret: când restaurant-service notifică proprietarul, apelează `http://tastetest-notification-service/api/notifications`. Segmentul de host este numele logic din Eureka, nu un IP. Spring Cloud LoadBalancer îl rezolvă la o instanță reală, iar dacă rulează mai multe copii ale serviciului, cererile alternează între ele (vezi §14, demo load balancing).

4. Componente de infrastructură

Trei servicii nu conțin logică de business, dar fac posibilă întreaga arhitectură distribuită: Discovery Server (cine există), Config Server (cu ce configurație) și API Gateway (cum intră lumea în sistem).

4.1 Discovery Server (Eureka)

Discovery Server rulează un **server Netflix Eureka** pe portul 8761. La pornire, fiecare micro-serviciu se înregistrează cu numele său logic (`spring.application.name`) și își anunță periodic disponibilitatea printr-un „heartbeat”. Celelalte servicii descarcă registrul și pot astfel rezolva un nume precum `tastetest-restaurant` la o adresă concretă.

Configurarea clientului din restul serviciilor folosește intervale scurte, pentru ca în mediul de dezvoltare schimbările de topologie să se propage rapid:

```
eureka:
  client:
    registry-fetch-interval-seconds: 5
  instance:
    lease-renewal-interval-in-seconds: 10
    lease-expiration-duration-in-seconds: 30
```

Rolul Discovery este esențial pentru două lucruri: **load balancing** (LoadBalancer alege o instanță din cele înregistrate sub un nume) și **reziliență** (dacă o instanță cade, lease-ul ei expiră și nu mai primește trafic).

4.2 Config Server

Config Server (port 8888) este un **Spring Cloud Config Server** care servește configurația din folderul `config-repo/`. La pornire, fiecare serviciu importă configurația de la el (`spring.config.import=optional:configserver:`) înainte să-și construiască propriul context.

Ce este externalizat în `config-repo/`:

- `token.secret` și `token.ttl` — secretul JWT și durata de viață a token-ului;
- credențialele inițiale de admin (folosite de `ApplicationInitializer`);
- setările `spring.datasource` și `spring.flyway` pentru profilul `dev`;
- host/port-ul Redis pentru `restaurant-service`;
- credențialele de mail.

Structura fișierelor urmează convenția Spring Cloud Config: `application.yml` (comun tuturor), `application-dev.yml` (comun pe profil), și câte un fișier per serviciu (`tastetest-restaurant.yml`, `tastetest-auth.yml`, ...) cu variantele `-dev` aferente.

Refresh dinamic fără restart

După modificarea unui fișier din `config-repo/`, configurarea poate fi reîncărcată la cald cu `POST /actuator/refresh` pe serviciul vizat. Beans-urile care consumă aceste valori sunt marcate cu `@RefreshScope`, deci se reconstruiesc cu noile valori fără a reporni procesul.

4.3 API Gateway

API Gateway (Spring Cloud Gateway, intrare publică pe `8090`) este singura ușă către sistem. Decide, pe baza căii din URL, către ce serviciu trimite cererea, și o face tot prin nume logic Eureka (`lb://...`), deci beneficiază de load balancing.

4.3.1 Tabela de rutare

Rută	Predicat (cale)	Destinație (lb)	Observații
auth-api	<code>/api/v1/auth/**</code>	<code>lb://tastetest-auth</code>	login, register, validare token
users-api	<code>/api/v1/user/**</code>	<code>lb://tastetest-user</code>	conturi, roluri, recenzii
restaurants-api	<code>/api/v1/restaurant/**</code>	<code>lb://tastetest-restaurant</code>	catalog, rating, meniuri
notificatiions-api	<code>/api/internal/notifications/**</code>	<code>lb://tastetest-notification-service</code>	rescrie calea în <code>/api/notifications...</code>
agent-api	<code>/api/v1/agent/**</code>	<code>lb://tastetest-agent</code>	declarată <i>înainte</i> de catch-all
backend-api	<code>/api/**</code> (catch-all)	<code>lb://tastetest-awdb</code>	monolitul legacy preia restul

Ordinea rutelor contează

Ruta `agent-api` (`/api/v1/agent/**`) este declarată **înainte** rutei catch-all `/api/**`. Spring Cloud Gateway evaluează rutele în ordine, iar prima care se potrivește câștigă. Dacă ordinea ar fi inversă, toate cererile către agent ar fi capturate de monolitul legacy.

4.3.2 Responsabilități transversale ale gateway-ului

Pe lângă rutare, gateway-ul concentrează câteva preocupări transversale, implementate ca filtre:

- **Validare JWT** (`JwtAuthenticationFilter`) — verifică semnătura token-ului la intrare, înainte ca cererea să ajungă la serviciul din spate.
- **Rate limiting** (`GatewayRateLimitFilter`) — limitează frecvența cererilor.
- **Propagarea anteturilor** (`GatewayHeadersFilter`) — adaugă anteturi precum `X-Gateway-Route` pentru trasabilitate.

- **Logarea cererilor** (`RequestLoggingFilter`) — jurnal centralizat al traficului.
- **CORS** (`CorsConfig`) — permite originea frontend-ului (`http://localhost:4200`).
- **Gestionarea erorilor** (`GlobalExceptionHandler`) — răspunsuri de eroare uniforme.

Timeout mărit pentru inferența AI

Apelurile către modelul LLM pot dura 60-120 de secunde. De aceea gateway-ul ridică explicit `spring.cloud.gateway.httpclient.response-timeout` la `150s`, ca să nu returneze `504 Gateway Timeout` înainte ca modelul să răspundă.

5. Microservicii de business

Aceasta este inima aplicației: cinci microservicii care țin logica de domeniu, plus monolitul original păstrat pentru migrare graduală. Fiecare are propriul controller, serviciu, repository și (unde e cazul) propria configurație de securitate și persistență.

5.1 auth-service — autentificare și emitere JWT

Responsabilitatea unică a acestui serviciu este **identitatea**: înregistrarea conturilor, autentificarea și emiterea token-urilor JWT. Este singurul serviciu care „semnează” token-uri; toate celelalte doar le validează.

5.1.1 Pachetul `com.example.auth`

Clasă	Rol
<code>controller/AuthController</code>	Expune <code>/api/v1/auth/register</code> , <code>/login</code> , <code>/token</code> .
<code>service/AuthService</code>	Logica de înregistrare (verifică email unic, encodează parola, atribuie rolul <code>USER</code>) și de login (autentică prin <code>AuthenticationManager</code> , generează token).
<code>config/security/JwtGenerator</code>	Construiește token-ul: subiect = email, claim <code>roles</code> , <code>issuedAt</code> / <code>expiration</code> , semnătură HS512.
<code>config/security/SecurityConfiguration</code>	Filtru JWT, sesiuni stateless, rute publice pentru login/register/swagger.
<code>config/ApplicationInitializer</code>	Creează contul de admin implicit la prima pornire (dezactivat pe profilul <code>test</code>).
<code>entity/User</code> , <code>entity/Role</code>	Entitățile relaționale mapate pe tabelele <code>users</code> , <code>roles</code> , <code>user_role</code> .

Cont admin implicit

La prima pornire pe profilul `dev`, `ApplicationInitializer` creează un cont de administrator: `username: admin`, `email: admin@admin.com`, `parola: admin`. Util pentru demo; de schimbat în producție.

5.2 user-service — conturi, roluri și recenzii

Gestionează partea de administrare a utilizatorilor și expune datele de recenzii pentru panoul de admin. Toate endpoint-urile sunt protejate la nivel de metodă cu `@PreAuthorize("hasAuthority('ADMIN')")`.

Endpoint	Rol	Descriere
GET /api/v1/user/all	ADMIN	Listează toți utilizatorii (DTO de admin).
GET /api/v1/user/{id}/reviews	ADMIN	Recenziile unui utilizator anume.
PUT /api/v1/user/{id}/roles	ADMIN	Actualizează lista de roluri a unui utilizator.

Serviciul are propriul stack de securitate (filtru JWT, `CustomUserDetailsService`) și propriile entități `User`, `Role`, `Review`. Recenziile sunt scrise din `user-service` — un detaliu important pentru caching (vezi §8): de aceea `restaurant-service` nu poate invalida cache-ul de rating local, ci se bazează pe TTL.

5.3 restaurant-service — catalog, meniuri și rating

Cel mai bogat serviciu de domeniu. Gestionează restaurantele, calculează rating-urile și orcheștrază saga de adăugare a unui restaurant. Este și singurul serviciu cu un **caching layer Redis**.

5.3.1 Endpoint-uri principale

Endpoint	Roluri	Descriere
POST /api/v1/restaurant/add	OWNER, ADMIN	Adaugă un restaurant (declanșează Saga).
PUT /api/v1/restaurant/update/{id}	OWNER, ADMIN	Editează un restaurant.
DELETE /api/v1/restaurant/delete/{id}	OWNER, ADMIN	Șterge un restaurant.
GET /api/v1/restaurant/get/{id}	toate	Detalii pentru un restaurant.
GET /api/v1/restaurant/getAll	toate	Toate restaurantele (cache 60s).
GET /api/v1/restaurant/paged	toate	Listă paginată (implicit 10/pagină, sortată după nume).
GET /api/v1/restaurant/top-rated	toate	Top după media rating-urilor (cache 30s).
GET /api/v1/restaurant/getRatings/{id}	toate	Media rating-urilor unui restaurant (cache 30s).
GET /api/v1/restaurant/getRestaurantId/{name}	toate	Rezolvă numele la UUID.

5.3.2 Componente notabile

- `saga/AddRestaurantSaga` — orchestratorul saga (persistă restaurant + notifică owner-ul).
- `saga/NotificationClient` — clientul REST către `notification-service`.

- `config/RestClientConfig` — `RestTemplate` balansat + interceptor care propagă antetul `Authorization`.
- `config/CacheConfig` — `@EnableCaching` + TTL per cache (Redis, serializare JSON).

5.4 notification-service — notificări și istoric NoSQL

Serviciu mic, dar cu două roluri importante: este **pasul 2 al saga** (notifică proprietarul la crearea unui restaurant) și este consumatorul bazei **NoSQL** a proiectului. Inițial era complet stateless; acum persistă fiecare notificare ca document în MongoDB.

Endpoint (intern, via gateway)	Descriere
POST <code>/api/notifications</code>	Primește o notificare, o salvează în Mongo și răspunde (cu <code>instanceId</code>).
GET <code>/api/notifications/history</code>	Ultimele 20 de notificări, ordonate descrescător după momentul procesării.

Documentul `NotificationLog` conține: titlu, mesaj, serviciul sursă, status, instanța care a procesat și timestamp. Câmpul `instanceId` din răspuns face vizibil ce instanță a tratat cererea — util pentru demo-ul de load balancing.

5.5 agent-service — asistentul AI (TasteBot)

Un microserviciu dedicat care găzduiește asistentul conversațional. Nu are bază de date proprie: este un **orchestrator peste serviciile existente**. Modelul LLM nu vede direct baza de date — are un set de *unelte* (tools) care împachetează endpoint-uri REST reale.

Unealtă (tool)	Endpoint din spate	Roluri
<code>listRestaurants</code>	GET <code>/restaurant/getAll</code>	USER, ADMIN, OWNER
<code>getTopRatedRestaurants</code>	GET <code>/restaurant/top-rated</code>	USER, ADMIN, OWNER
<code>getRestaurantDetails(id)</code>	GET <code>/restaurant/get/{id}</code>	USER, ADMIN, OWNER
<code>getAverageRating(id)</code>	GET <code>/restaurant/getRatings/{id}</code>	USER, ADMIN, OWNER
<code>findRestaurantIdByName(name)</code>	GET <code>/restaurant/getRestaurantId/{name}</code>	USER, ADMIN, OWNER
<code>listAllUsers</code>	GET <code>/user/all</code>	doar ADMIN
<code>getUserReviews(userId)</code>	GET <code>/user/{id}/reviews</code>	doar ADMIN

Trei garanții de design

Date reale, fără fabricație — modelul nu poate inventa restaurante, pentru că uneltele merg mereu la catalogul real. **Autorizarea se păstrează** — uneltele de admin întorc 403 pentru utilizatori obișnuiți, iar modelul comunică asta. **Fără bază de date nouă** — agentul este pur orchestrator.

5.6 backend-original — monolitul legacy

Versiunea monolitică inițială a aplicației, păstrată intenționat în arhitectură. Este **internă** (nu are port expus direct) și preia, prin ruta catch-all `/api/**` a gateway-ului, funcționalitățile care încă nu au fost migrate în microservicii dedicate (meniuri, articole, categorii, răspunsuri, feedback). Reprezintă strategia de migrare graduală „strangler fig”: microserviciile noi preiau treptat responsabilități, iar monolitul rămâne ca plasă de siguranță până când toate domeniile sunt extrase.

6. Modelul de date

Datele relaționale trăiesc într-o schemă PostgreSQL numită `project`, partajată de serviciile care au nevoie de ea. Pe lângă aceasta, proiectul folosește două depozite NoSQL: Redis pentru cache și MongoDB pentru istoricul de notificări. Schema relațională este gestionată versionat prin migrări Flyway.

6.1 Diagrama entitate-relație (ERD)

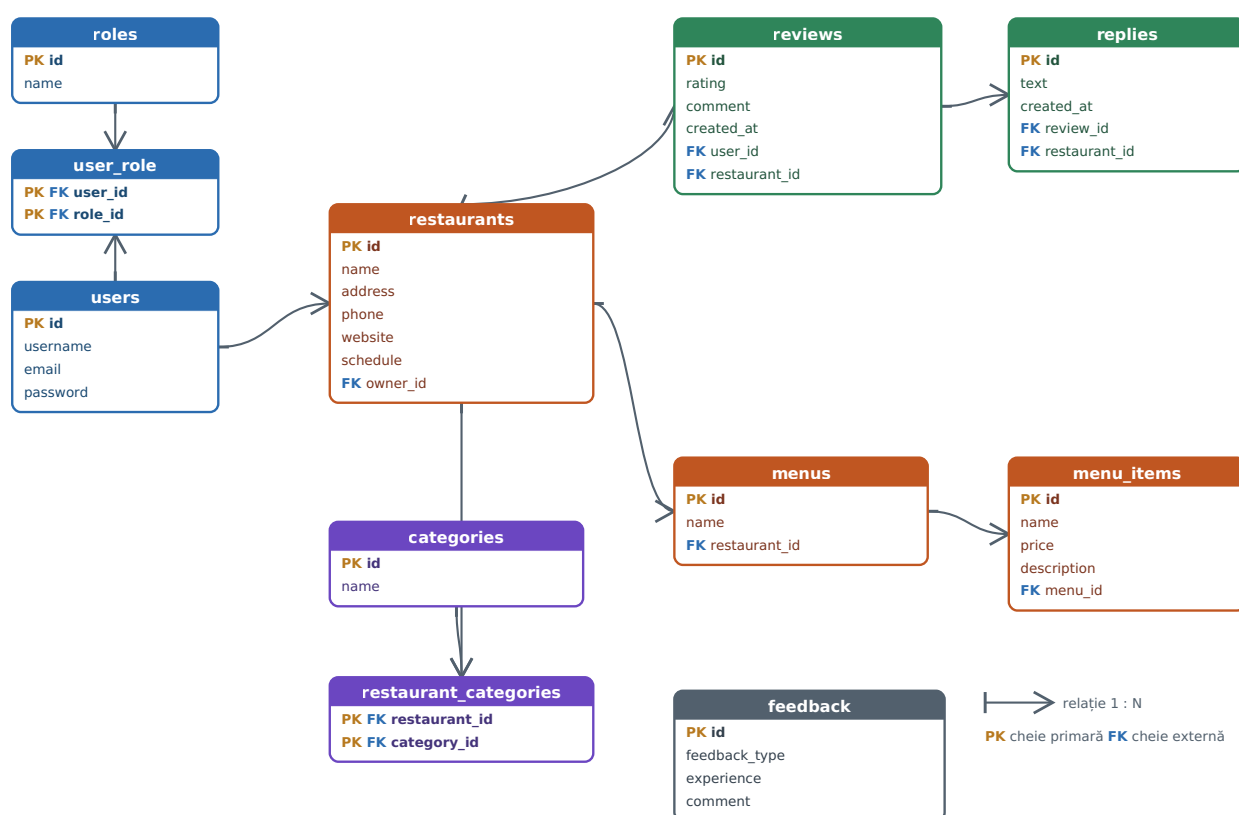


Figura 6.1 — Modelul relațional al schemei `project`: 11 tabele, cu chei primare (PK) și externe (FK). Relațiile sunt 1:N; `user_role` și `restaurant_categories` sunt tabele de legătură pentru relațiile many-to-many.

6.2 Descrierea tabelelor

Tabel	Coloane-cheie	RoI
users	id (UUID, PK), username, email (unic), password (hash)	Conturile utilizatorilor.
roles	id (int, PK), name (unic)	Rolurile fixe: ADMIN, USER, RESTAURANT_OWNER.
user_role	user_id + role_id (PK compus, FK)	Legătura M:N user-rol.
restaurants	id (PK), name (unic), address, phone, website, schedule, owner_id (FK→users)	Restaurantele și proprietarul lor.
reviews	id (PK), rating (1-5), comment, created_at, user_id (FK), restaurant_id (FK)	Recenziile utilizatorilor.
replies	id (PK), text, created_at, review_id (FK), restaurant_id (FK)	Răspunsurile restaurantului la recenzii.
menus	id (PK), name, restaurant_id (FK)	Meniurile unui restaurant.
menu_items	id (PK), name, price (DECIMAL), description, menu_id (FK)	Articolele dintr-un meniu.
categories	id (PK), name (unic)	Categoriile culinare.
restaurant_categories	restaurant_id + category_id (PK compus, FK)	Legătura M:N restaurant-categorie.
feedback	id (PK), feedback_type, experience, comment	Feedback general despre platformă (fără FK — poate fi anonim).

6.3 Relații și integritate referențială

Relațiile sunt protejate prin chei externe, majoritatea cu `ON DELETE CASCADE`, ceea ce asigură curățarea automată a datelor dependente:

- **users ↔ roles** (M:N prin `user_role`) — un utilizator poate avea mai multe roluri.
- **users → restaurants** (1:N, `owner_id`) — ștergerea unui utilizator șterge restaurantele lui (cascade).
- **users → reviews** și **restaurants → reviews** (1:N) — o recenzie aparține unui utilizator și unui restaurant.
- **reviews → replies** și **restaurants → replies** (1:N) — un răspuns e legat de o recenzie și de restaurant.
- **restaurants → menus → menu_items** (lanț 1:N) — meniurile și articolele se șterg odată cu restaurantul.
- **restaurants ↔ categories** (M:N prin `restaurant_categories`).

Validările la nivel de coloană includ: `email` unic pe `users`, `name` unic pe `restaurants` și `categories`, și constrângerea `CHECK (rating BETWEEN 1 AND 5)` pe `reviews`.

6.4 Migrări Flyway

Schema este construită prin migrări versionate Flyway, rulate automat la pornire pe profilul `dev`. Migrările monolitului (V1-V13) construiesc întreaga schemă și introduc date de test; migrările per-serviciu (V14-V16) sunt **idempotente** (`CREATE TABLE IF NOT EXISTS`), astfel încât fiecare microserviciu să-și poată asigura tabelele de care are nevoie, indiferent de ordinea pornirii.

Migrare	Conținut
V1__INIT	Schema <code>project</code> , tabelele <code>users</code> , <code>roles</code> , <code>user_role</code> ; inserează cele 3 roluri.
V2__CREATE_RESTAURANT	Tabelul <code>restaurants</code> + FK către <code>users</code> .
V3__CREATE_REVIEWS	Tabelul <code>reviews</code> cu <code>CHECK</code> pe <code>rating</code> .
V4__CREATE_REPLY	Tabelul <code>replies</code> .
V5__CREATE_MENU	Tabelele <code>menus</code> și <code>menu_items</code> .
V6__CREATE_CATEGORY	Tabelele <code>categories</code> și <code>restaurant_categories</code> .
V7-V11	Date de test: utilizatori, restaurante, recenzii, răspunsuri, meniuri.
V12__CREATE_FEEDBACK	Tabelul <code>feedback</code> .
V13__ADD_MORE_RESTAURANTS	Date suplimentare de catalog.
V14 / V15 / V16	Migrări idempotente per-serviciu (<code>user</code> / <code>auth</code> / <code>restaurant</code>) — asigură tabelele necesare fiecărui microserviciu.

Decizie: bază de date partajată vs. una per serviciu

Manualul „pur” de microservicii recomandă o *bază de date per serviciu*. Aici, serviciile partajează schema `project` din PostgreSQL. Este un compromis pragmatic pentru contextul academic: simplifică integritatea referențială (FK reale între restaurante, utilizatori și recenzii) și evită sincronizarea cross-DB. Prețul plătit este cuplarea la nivel de schemă — vizibilă, de exemplu, în faptul că `restaurant-service` nu poate invalida cache-ul de `rating` când o recenzie e scrisă din `user-service` (vezi §8.2).

6.5 Depozite NoSQL

MongoDB este baza NoSQL a proiectului, folosită de `notification-service`. Colecția `notifications` stochează câte un document `NotificationLog` pentru fiecare notificare procesată, cu câmpurile: titlu, mesaj, serviciu sursă, status, instanță și timestamp. Notificările sunt înregistrări

de tip „append-only”, cu schemă lejeră — un caz natural pentru un document store, nu pentru încă un tabel în PostgreSQL.

Redis nu este o bază de date de domeniu, ci un strat de cache cheie-valoare în memorie pentru restaurant-service (detaliat în §8.2). Datele sunt efemere (TTL) și pot fi reconstruite oricând din PostgreSQL.

7. Securitate și autorizare

Securitatea se bazează pe token-uri JWT stateless. auth-service este singura autoritate care emite token-uri; toate celelalte servicii le validează local, fără apeluri suplimentare, folosind același secret partajat. Autorizarea se face pe roluri, la nivel de endpoint.

7.1 Modelul: JWT stateless + secret partajat

Token-ul este un JWT semnat cu algoritmul **HS512** (HMAC-SHA512). El conține: subiectul (email-ul utilizatorului), un claim `roles` cu lista rolurilor, momentul emiterii, expirarea și emitentul. Pentru că semnătura folosește un secret simetric partajat de toate serviciile (injectat prin Config Server / variabile de mediu ca `token.secret`), orice serviciu poate verifica autenticitatea token-ului independent — nu există o sesiune server-side și niciun apel către auth-service la fiecare cerere.

```
// JwtGenerator (auth-service) – semnarea token-ului
Jwts.builder()
    .setSubject(username)           // email
    .claim("roles", roles)         // ex. ["USER"]
    .setIssuedAt(now).setExpiration(exp) // TTL configurat
    .setIssuer("http://localhost:8090")
    .signWith(signingKey, SignatureAlgorithm.HS512)
    .compact();
```


7.2 Fluxul token-ului end-to-end

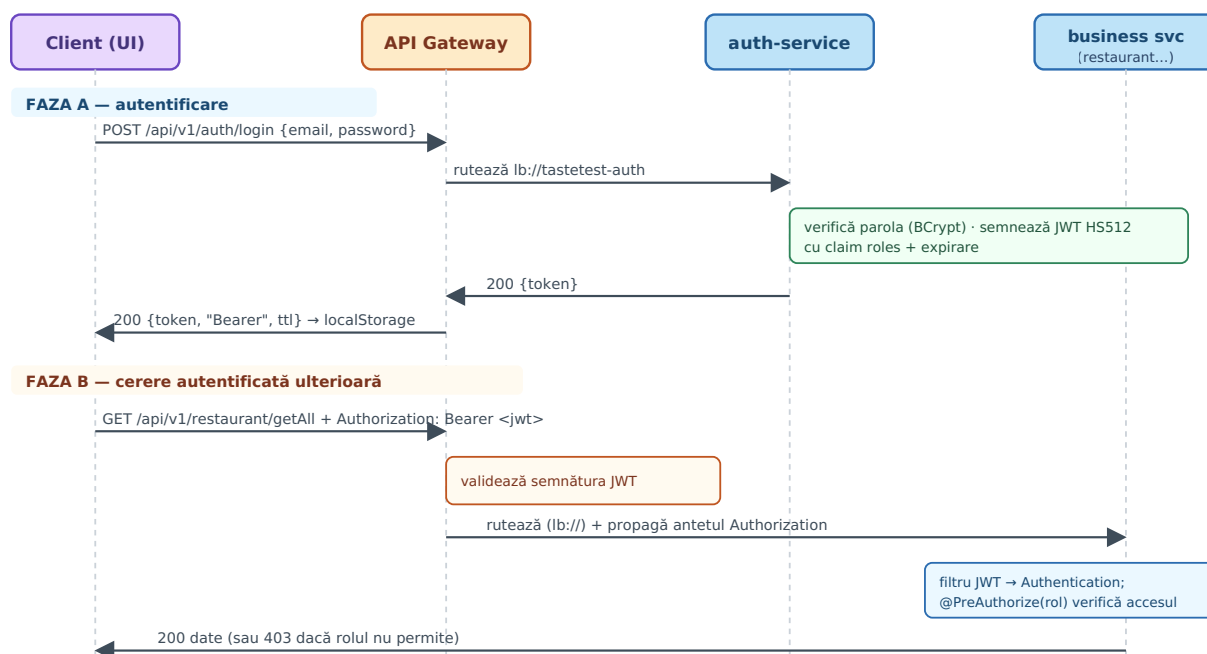


Figura 7.1 — Fluxul JWT: faza de autentificare (A) produce token-ul, iar fiecare cerere ulterioară (B) îl prezintă, fiind validat la gateway și reverificat în serviciul de destinație pentru autorizare pe rol.

7.3 Roluri și autorizare

Există trei roluri, atribuite prin tabelul `user_role`. La înregistrare, orice cont primește implicit rolul `USER`. Autorizarea se aplică declarativ, cu `@PreAuthorize` pe metodele de controller și `@EnableMethodSecurity` pe configurația de securitate.

Rol	Poate face
USER	Vede restaurante/meniuri/recenzii, lasă recenzii, trimite feedback, folosește asistentul AI.
RESTAURANT_OWNER	Tot ce poate USER + adaugă/editează/șterge restaurante și meniuri, răspunde la recenzii.
ADMIN	Acces complet: gestionează utilizatori și roluri, vede toate recenziile, administrează feedback și catalog.

7.4 Configurația de securitate per serviciu

Fiecare serviciu de business are aceeași schemă de securitate: CSRF dezactivat (API stateless), sesiuni STATELESS, un `JwtAuthenticationFilter` adăugat înaintea filtrului standard de username/parolă, și o listă de rute publice (login, register, `/actuator/**`, Swagger). Toate celelalte cereri necesită autentificare.

Comutator de securitate pentru teste

Securitatea poate fi dezactivată complet prin `vars.security.enable: false`. Pe profilul `test` acest comutator este oprit, ca testele și demo-urile rapide să nu necesite token-uri. În `dev` / producție securitatea este activă implicit.

8. Decizii arhitecturale (ADR)

Această secțiune adună deciziile semnificative de proiectare, fiecare cu **contextul** (ce problemă rezolvă), **decizia** (ce s-a ales) și **consecințele** (ce câștigăm și ce plătim). Sunt deciziile care diferențiază proiectul de un simplu CRUD.

8.1 ADR-1: Pattern Saga (orchestrare) pentru adăugarea unui restaurant

Context. Crearea unui restaurant este o mică tranzacție distribuită: (1) se persistă restaurantul în PostgreSQL-ul lui restaurant-service și (2) se notifică proprietarul prin notification-service, peste granița de serviciu. O singură tranzacție JPA nu poate înveli atomic ambii pași: dacă apelul de notificare eșuează după ce rândul a fost deja comis, am rămâne cu un restaurant „viu” despre care proprietarul nu a fost niciodată anunțat — un succes parțial tăcut.

Decizie. Se folosește pattern-ul **Saga prin orchestrare**: un orchestrator (`AddRestaurantSaga`) execută pașii în ordine, fiecare cu propria tranzacție; dacă un pas ulterior eșuează, rulează acțiuni de compensare care anulează pașii deja comiși.

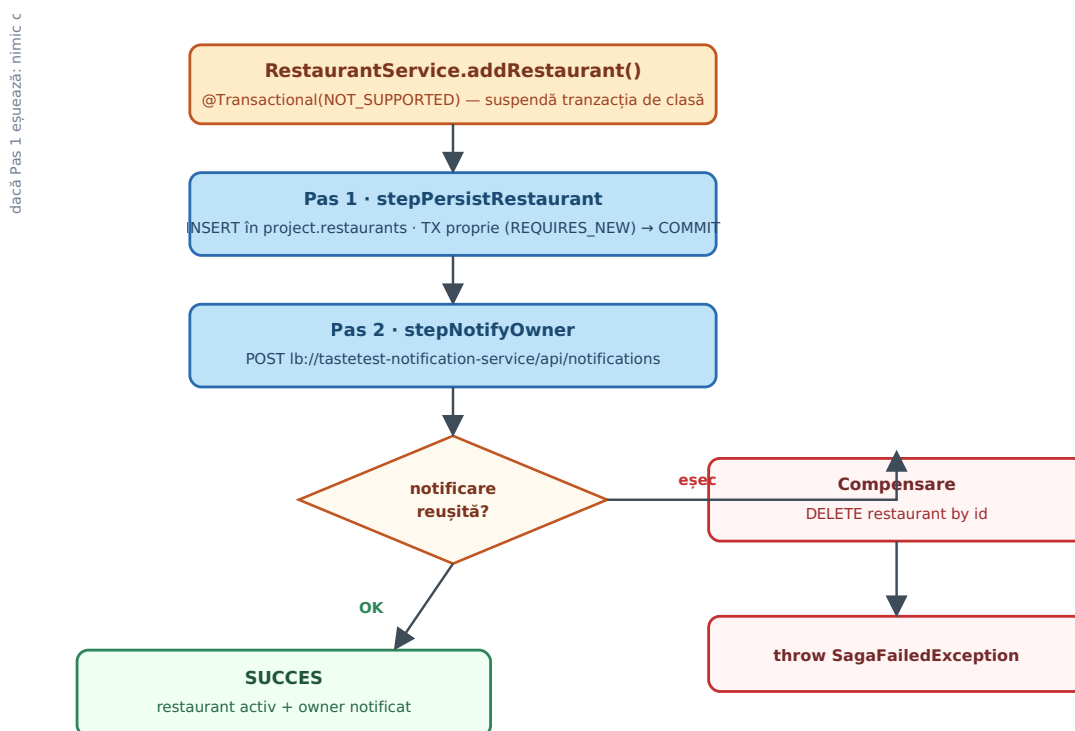


Figura 8.1 — Saga de adăugare a restaurantului. Fiecare pas comite într-o tranzacție proprie (REQUIRES_NEW); eșecul pasului 2 declanșează compensarea pasului 1.

Detaliu de implementare. Saga rulează deliberat în afara oricărei tranzacții JPA: metoda este `@Transactional(NOT_SUPPORTED)`, iar fiecare pas folosește un `TransactionTemplate` cu `PROPAGATION_REQUIRES_NEW`. Asta face compensarea semnificativă — altfel, o scriere necomisă s-ar anula

singură prin rollback JPA și nu ar exista nimic de „compensat”. Apelul de notificare propagă an-tetul `Authorization` al utilizatorului curent, deci este autentificat ca același end-user.

Punct de eșec	Rezultat
Pas 1 (DB) aruncă	Nimic comis → fără compensare. Saga ridică <code>SagaFailedException</code> .
Pas 2 (notificare) aruncă	Rulează compensarea: rândul restaurantului e șters. Apelantul vede <code>SagaFailedEx-ception</code> .
Compensarea însăși eșuează	Logat la ERROR; sistemul rămâne inconsistent → necesită reconciliere manuală (limita cunoscută a sagăi).

De ce orchestrare, nu coregrafie. Coregrafia (servicii care reacționează la evenimente pe un broker Kafka/RabbitMQ) ar fi adăugat un broker pe care proiectul nu îl are. Cu doar doi pași și o compensare, orchestrarea sincronă peste REST+Eureka este suficientă și mult mai ușor de urmărit. Dacă saga ar crește peste ~3 pași sau ar avea nevoie de retry cu backoff, coregrafia pe broker ar deveni alegerea potrivită.

8.2 ADR-2: Caching (Redis) + NoSQL (MongoDB)

Context. Trei căi de citire din restaurant-service sunt disproporționat de scumpe față de cât de rar se schimbă datele — în special `top-rated`, care încărca toate restaurantele și, pentru fieca-re, toate recenziile, ca să calculeze media (un clasic $N+1$). Separat, notification-service era complet stateless și nu păstra niciun istoric.

Decizie. Se introduce un **caching layer Redis** pentru cele trei citiri fierbinți, cu TTL diferențiat, și o **bază NoSQL MongoDB** pentru istoricul de notificări.

Metodă	Cache	TTL	Invalidare
<code>getAllRestaurants()</code>	<code>allRestaurants</code>	60s	<code>@CacheEvict</code> la add/update/delete (local)
<code>getTopRatedRestaurants()</code>	<code>topRated</code>	30s	parțial evict local + TTL pentru recenzii
<code>getRatings(id)</code>	<code>ratings (per id)</code>	30s	doar TTL

De ce TTL în loc de invalidare explicită pentru rating

Rating-urile depind de **recenzii**, care se scriu din user-service — alt JVM, cu alt context de cache. Nu există un eveniment in-process pe care să atârni un `@CacheEvict` când o recenzie e creată acolo (ar necesita un broker / eveniment de invalidare, în afara scopului). Așadar: pentru CRUD-ul de restaurant, care este local, se folosește invalidare explicită; pentru da-tele derivate din recenzii, se acceptă o consistență eventuală — o recenzie nouă poate dura până la 30s să se reflecte în `top-rated`. Este un compromis documentat, nu o scăpa-re.

De ce MongoDB aici. Notificările sunt înregistrări append-only, cu schemă lejeră — potrivite pentru un document store, nu pentru încă un tabel relațional. Este o bază genuin separată (con-

tainer și volum proprii), demonstrând persistență poliglotă reală, nu reutilizarea PostgreSQL-ului existent.

8.3 ADR-3: Configurare centralizată (Config Server)

Context. Secretele (JWT, mail) și parametrii de mediu (datasource, Redis) erau presărați prin fișierele de configurare ale fiecărui serviciu. **Decizie.** Externalizarea lor în `config-repo/`, servită de un Config Server, cu refresh la cald prin `@RefreshScope` + `POST /actuator/refresh`. **Consecințe.** O singură sursă de adevăr pentru configurare, secrete scoase din codul serviciilor, posibilitatea de a schimba valori fără rebuild; în schimb, un serviciu suplimentar de care depind toate celelalte la pornire (atenuat prin `optional:configserver:` și healthcheck-uri în Compose).

8.4 ADR-4: Service Discovery + Load Balancing client-side

Context. Microserviciile au nevoie să se găsească reciproc fără adrese hardcodate, iar unele rulează în mai multe instanțe. **Decizie.** Eureka pentru descoperire + Spring Cloud LoadBalancer pentru balansare client-side (apeluri `lb://nume-serviciu`). **Consecințe.** Topologie dinamică (instanțele pot apărea/disparea), balansare fără un load balancer dedicat; în schimb, fiecare client trebuie să integreze logica de LB, iar registrul devine o componentă critică.

8.5 ADR-5: API Gateway ca punct unic de intrare

Context. Expunerea directă a fiecărui microserviciu ar fi împrăștiat CORS, autentificarea și rate limiting-ul peste tot. **Decizie.** Un singur Spring Cloud Gateway concentrează rutarea, validarea JWT, CORS, rate limiting și logarea. **Consecințe.** Clienții văd un singur host; preocupările transversale stau într-un loc; în schimb, gateway-ul devine un punct unic prin care trece tot traficul (atenuat prin faptul că e stateless și scalabil).

8.6 ADR-6: Micro-frontends prin iframe

Context. Panoul de admin are public diferit, stare separată și cadență de release plauzibil diferită față de restul UI-ului. **Decizie.** Extragerea lui într-un workspace Angular independent (`mfe-admin`), compus în shell la runtime printr-un **iframe**, cu predarea token-ului prin fragmentul de URL (`#token=...`).

De ce iframe, nu Module Federation / Angular Elements

Izolare totală — fără ciocniri de Zone.js sau injector între două runtime-uri Angular în același DOM. **Cu adevărat poliglot** — MFE-ul ar putea fi rescris mâine în React/Vue fără schimbări în shell. **Trivial de verificat** — `<iframe src=URL>` e un contract pe care orice evaluator îl poate inspecta. **Fragmentul de URL (#), nu query param** — fragmentele nu se trimit la server, deci token-ul nu ajunge în loguri nginx sau anteturi `Referer`.

8.7 ADR-7: Agent AI ca microserviciu-orchestrator

Context. Un asistent conversațional trebuie să răspundă pe baza datelor reale, nu să le inventeze, și să respecte autorizarea. **Decizie.** Un microserviciu dedicat (agent-service) cu Spring AI, în care modelul are *unelte* ce împachetează endpoint-uri REST reale; uneltele propagă token-ul JWT al utilizatorului, deci autorizarea se păstrează natural. **Consecințe.** Date me-reu reale, autorizare respectată, fără bază de date nouă; în schimb, latență mare la inferență (de aici timeout-ul de 150s din gateway) și dependență de un model extern.

9. Fluxuri de utilizare

Această secțiune urmărește principalele scenarii end-to-end, de la acțiunea utilizatorului în UI până la efectele din backend, arătând cum colaborează serviciile descrise mai sus.

9.1 Înregistrare cont

1. Utilizatorul completează formularul de la `/register` (username, email, parolă).
2. Frontend-ul trimite `POST /api/v1/auth/register` prin gateway către auth-service.
3. auth-service verifică dacă email-ul este deja folosit; dacă da, întoarce `400 „Email is already used”`.
4. Parola este encodată (BCrypt), contul este salvat cu rolul implicit `USER`.
5. Se întoarce `201 „User registered”`; utilizatorul este direcționat spre login.

9.2 Autentificare

1. Utilizatorul completează `/login` (email, parolă).
2. `POST /api/v1/auth/login` → auth-service autentifică prin `AuthenticationManager`.
3. La succes, `JwtGenerator` emite un JWT HS512 cu claim-ul `roles`.
4. Răspunsul `{ token, "Bearer", ttl }` este stocat în `localStorage.access_token`.
5. Un HTTP interceptor adaugă automat `Authorization: Bearer <token>` la toate cererile ulterioare.

9.3 Adăugarea unui restaurant (cu Saga)

Un `RESTAURANT_OWNER` sau `ADMIN` completează formularul de restaurant. `POST /api/v1/restaurant/add` declanșează saga descrisă în §8.1: restaurantul este persistat (pas 1), apoi proprietarul este notificat prin notification-service (pas 2). Dacă notificarea eșuează, restaurantul este șters prin compensare și utilizatorul primește eroare; altfel, restaurantul devine vizibil în catalog, iar notificarea este salvată în MongoDB.

9.4 Lăsarea unei recenzii

1. Un `USER` autentificat deschide detaliile unui restaurant (`/restaurants/:id`) și apasă „Adaugă recenzie”.
2. Completează nota (1-5 stele) și un comentariu, apoi trimite formularul de la `/restaurants/:id/review`.
3. Recenzia este persistată legată de utilizator și de restaurant.

- Media rating-ului restaurantului se va actualiza în `top-rated / getRatings` în maximum 30s (expirarea cache-ului Redis — vezi §8.2).

9.5 Răspuns la o recenzie (owner)

Reprezentantul restaurantului vede recenziile primite și poate adăuga un răspuns (`reply`), legat de recenzie și de restaurant. Astfel se încheie bucla de comunicare client-restaurant, vizibilă public în pagina de detalii.

9.6 Conversație cu asistentul AI (TasteBot)

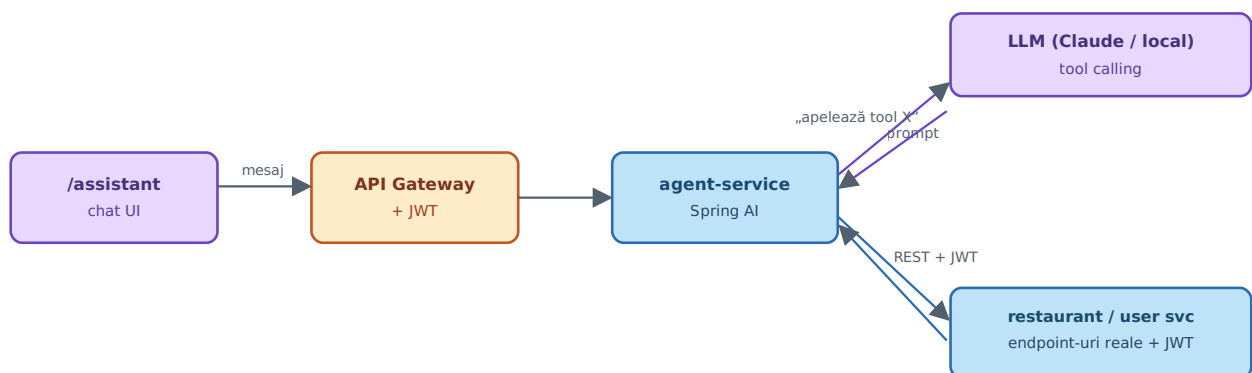


Figura 9.1 — Buclea agentului: modelul decide ce unealtă să apeleze, agent-service execută apelul REST real (cu JWT-ul utilizatorului), iar rezultatul reintră în raționamentul modelului.

- Utilizatorul autentificat scrie o întrebare în `/assistant` (ex. „Care e cel mai bine cotate restaurant?”).
- `POST /api/v1/agent/chat` ajunge la agent-service prin gateway (token-ul este verificat).
- Modelul decide să apeleze o unealtă (ex. `getTopRatedRestaurants`); agent-service face apelul REST real către restaurant-service, propagând JWT-ul.
- Rezultatul reintră în model, care formulează un răspuns conversațional, întors în UI.
- Dacă o unealtă necesită ADMIN și utilizatorul nu are rolul, apelul întoarce 403, iar modelul explică limitarea în loc să inventeze.

9.7 Administrare (admin)

- Un `ADMIN` autentificat navighează la `/admin` în shell.
- Shell-ul randează un `iframe` către micro-frontend-ul de admin (`http://localhost:4201/#token=<jwt>`).
- MFE-ul preia token-ul din fragment, îl stochează local, curăță URL-ul și pornește aplicația de admin.

4. Adminul gestionează utilizatori și roluri (`/api/v1/user/*`), vede recenzii și administrează feedback.

10. Frontend (Angular)

Interfața este împărțită în două aplicații Angular 21 independente: shell-ul principal (`frontend/`) și micro-frontend-ul de admin (`mfe-admin/`). Ambele folosesc componente standalone, un interceptor JWT și sunt servite în producție de nginx.

10.1 Shell-ul principal

Shell-ul conține tot UI-ul orientat spre utilizator: pagina de start, autentificare, catalogul de restaurante, detalii, formulare, recenzii, feedback și asistentul AI. Comunică cu backend-ul prin `/api/v1` (proxy în dezvoltare, nginx în producție).

10.1.1 Structura

Director	Conținut
<code>models/</code>	Interfețe TypeScript pentru DTO-uri (auth, restaurant, review, menu, feedback, reply, page).
<code>services/</code>	Câte un serviciu per entitate (Auth, Restaurant, Review, Menu, MenuItem, Reply, Feedback, User, Role, Agent).
<code>interceptors/</code>	<code>auth.interceptor</code> — adaugă antetul Bearer la fiecare cerere.
<code>guards/</code>	<code>auth.guard</code> — protejează rutele care necesită autentificare.
<code>components/</code>	<code>navbar</code> , <code>home</code> , <code>login</code> , <code>register</code> , <code>restaurants</code> , <code>restaurant-detail</code> , <code>restaurant-form</code> , <code>review-form</code> , <code>menu-detail</code> , <code>menu-form</code> , <code>menu-item-form</code> , <code>feedback-form</code> , <code>assistant</code> , <code>admin</code> , <code>not-found</code> .

10.1.2 Rutele aplicației

Rută	Componentă	Protejată
/home, /login, /register	Home / Login / Register	nu
/restaurants	listă restaurante	da
/restaurants/:id	detalii restaurant	da
/restaurants/add, /restaurants/:id/edit	formular restaurant	da
/restaurants/:id/review	formular recenzie	da
/menus/:id, /menus/:id/edit, /menus/:menuId/items/add	meniuri și articole	da
/feedback	formular feedback	da
/assistant	TasteBot (chat)	da
/admin	iframe MFE admin	da
/**	NotFound (404)	nu

10.2 Micro-frontend-ul de admin

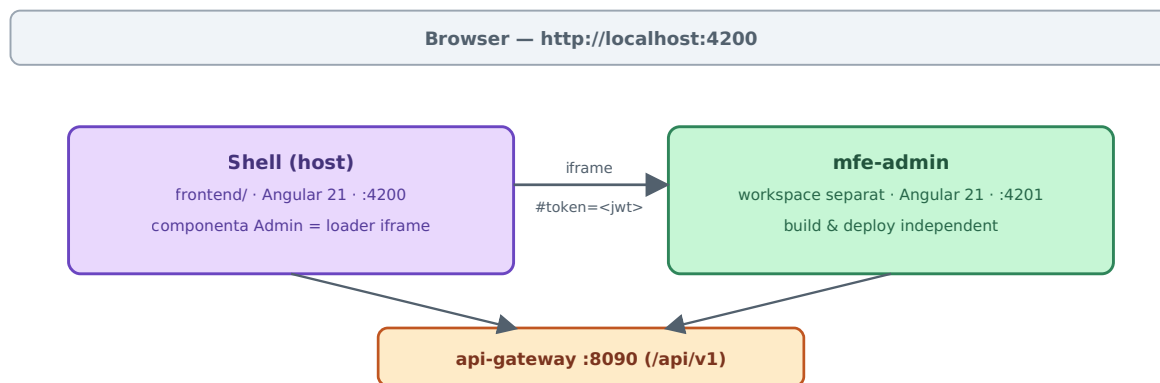


Figura 10.1 — Compoziția micro-frontend: shell-ul încarcă MFE-ul de admin printr-un iframe, predând token-ul prin fragmentul de URL. Ambele apelează același gateway.

Panoul de admin a fost extras într-un workspace Angular complet separat: `package.json` și `angular.json` proprii, `Dockerfile` multi-stage propriu (node→nginx) și port publicat separat (4201). Shell-ul știe despre MFE doar două lucruri: URL-ul lui public și contractul de autentificare (token-ul prin fragment). Modelele și serviciile sunt duplicate intenționat — acesta este prețul independenței, și exact ce face fiecare parte înlocuibilă fără a o atinge pe cealaltă.

Izolarea defectelor

Dacă build-ul de admin se strică, utilizatorii obișnuiți continuă să funcționeze: `/admin` arată un `iframe` rupt, dar restul aplicației nu este afectat. Oprirea containerului `mfe-admin` nu doboară shell-ul — o demonstrație clară a izolării.

11. Referință API

Toate endpoint-urile sunt accesate public prin API Gateway pe `http://localhost:8090`. Gateway-ul rutează pe baza prefixului căii către serviciul potrivit. Mai jos, endpoint-urile sunt grupate pe serviciu, cu metoda, calea, rolurile necesare și o scurtă descriere.

GET **POST** **PUT** **DELETE** — metodele HTTP. **ROL** indică autoritatea necesară.

11.1 auth-service ▪ `/api/v1/auth`

Metodă	Cale	Roluri	Descriere
POST	<code>/register</code>	public	Înregistrează un cont nou (primește rolul USER).
POST	<code>/login</code>	public	Autentifică și întoarce un token JWT.
GET	<code>/token</code>	autentificat	Validează token-ul curent și întoarce email-ul.

11.2 user-service ▪ `/api/v1/user`

Metodă	Cale	Roluri	Descriere
GET	<code>/all</code>	ADMIN	Listează toți utilizatorii.
GET	<code>/{"id"}/reviews</code>	ADMIN	Recenziile unui utilizator.
PUT	<code>/{"id"}/roles</code>	ADMIN	Actualizează rolurile unui utilizator.

11.3 restaurant-service ▪ /api/v1/restaurant

Metodă	Cale	Roluri	Descriere
POST	/add	OWNER, ADMIN	Adaugă un restaurant (Saga).
PUT	/update/{id}	OWNER, ADMIN	Editează un restaurant.
DELETE	/delete/{id}	OWNER, ADMIN	Șterge un restaurant.
GET	/get/{id}	toate	Detalii pentru un restaurant.
GET	/getAll	toate	Toate restaurantele (cache 60s).
GET	/paged	toate	Listă paginată (10/pagină, sortată după nume).
GET	/top-rated	toate	Top după rating mediu (cache 30s).
GET	/getRatings/{id}	toate	Rating mediu al unui restaurant (cache 30s).
GET	/getRestaurantId/{name}	toate	Rezolvă numele la UUID.

„toate” = USER, RESTAURANT_OWNER și ADMIN.

11.4 agent-service ▪ /api/v1/agent

Metodă	Cale	Ro-luri	Descriere
POST	/chat	toate	Trimite un mesaj asistentului AI; răspunsul folosește tool calling peste serviciile reale.

11.5 notification-service ▪ /api/internal/notifications

Rutat prin gateway, care rescrie calea în /api/notifications... către serviciu.

Metodă	Cale	Roluri	Descriere
POST	/	autentificat	Înregistrează o notificare (salvată în MongoDB).
GET	/history	autentificat	Ultimele 20 de notificări.

11.6 backend-original ▪ /api/** (catch-all)

Domeniile încă nemigrate (meniuri, articole de meniu, categorii, răspunsuri la recenzii, feedback) sunt servite de monolit prin ruta catch-all a gateway-ului. Endpoint-urile respectă același prefix /api/v1/... și aceeași schemă de autorizare pe roluri.

11.7 Actuator (toate serviciile)

Endpoint	Descriere
GET /actuator/health	Starea serviciului (UP/DOWN).
GET /actuator/metrics	Metriци (CPU, memorie, request rate).
GET /actuator/prometheus	Metriци în format Prometheus (scrape).
POST /actuator/refresh	Reîncarcă configurarea din Config Server.

12. Observabilitate și monitorizare

Stack-ul de observabilitate are trei piese: Spring Boot Actuator pe fiecare serviciu (sursa metricilor), Prometheus (colectare), Grafana (vizualizare) și Zipkin (tracing distribuit).

12.1 Componente

Componentă	Acces	Rol
Actuator	/actuator/*	Expune health, metrics, info, prometheus pe fiecare serviciu.
Prometheus	:9090	Face scrape periodic la /actuator/prometheus și stochează seriile temporale.
Grafana	:3000 (admin/admin)	Dashboard „TasteTest Observability”: CPU, memorie heap JVM, request rate.
Zipkin	:9411	Tracing distribuit (Micrometer Tracing + Brave) — urmărește o cerere prin servicii.

12.2 Interogări Prometheus utile

```
up # ce ținte sunt sus
process_cpu_usage # CPU per proces
jvm_memory_used_bytes # memorie JVM
rate(http_server_requests_seconds_count[5m]) # request rate
```

În `Status → Targets` ar trebui să apară toate job-urile serviciilor Spring în starea `UP`. Dacă Zipkin nu arată trace-uri, de obicei înseamnă trafic insuficient sau spans negenerate încă.

12.3 Flux rapid pentru demonstrație

1. Deschide `/actuator/health` pe un serviciu și arată starea `UP`.
2. În Prometheus, verifică `Status → Targets`.
3. În Grafana, arată dashboard-ul cu CPU, memorie și request rate.
4. Generează câteva cereri, apoi caută trace-urile în Zipkin.

13. CI/CD

Pipeline-ul de integrare și livrare continuă rulează în GitHub Actions (`.github/workflows/ci.yml`), la fiecare push și pull request pe `main` . Acoperă build automat, teste automate, containerizare Docker și deployment automat în staging.

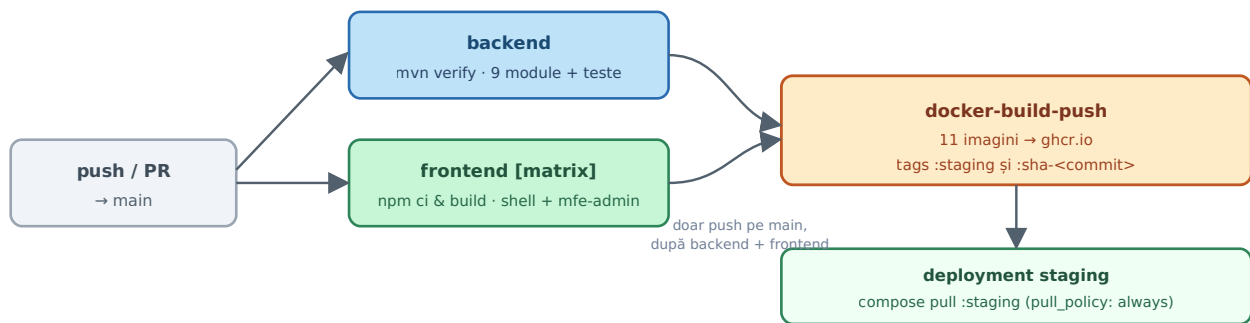


Figura 13.1 — Pipeline-ul CI/CD: build backend + frontend rulează la fiecare push/PR; publicarea imaginilor Docker se face doar la push pe main, după ce primele două etape trec.

13.1 Etapele pipeline-ului

1. **backend** — `./mvnw -B -ntp verify` la rădăcină: compilează toate cele 9 module ale reactorului Maven și rulează testele unitare existente (ex. `ReviewServiceTest` , un test Mockito pur, fără dependențe externe).
2. **frontend** — job matrix peste `frontend` și `mfe-admin`: fiecare rulează `npm ci + npm run build` cu Node 20.
3. **docker-build-push** — doar la push pe `main` , după ce primele două trec: construiește cele 11 imagini și le publică în GitHub Container Registry cu tag-urile `:staging` (ultimul build de pe main) și `:sha-<commit>` (imutabil, trasabil).

Publicarea unei imagini `:staging` noi este pasul de deployment automat: un mediu de staging care rulează `docker compose` cu `image: ...:staging` și `pull_policy: always` preia noul build la următorul `docker compose pull && up -d` .

14. Rulare, profiluri și deployment

Proiectul poate fi rulat de la un singur serviciu Spring până la întregul stack de 16+ containere, în funcție de scenariu. Două profiluri Spring separă mediul de dezvoltare de cel de testare.

14.1 Profiluri Spring

Profil	Bază de date	Port	Flyway	DDL	Securitate	Use case
dev	PostgreSQL (Docker)	8090	activat	none	activă	dezvoltare zilnică
test	H2 in-memory	8091	dezactivat	create-drop	dezactivată	teste, demo rapid fără Docker

14.2 Cerințe pentru rulare locală

- **Java 21** (proiectul folosește features moderne);
- **Maven Wrapper** (`./mvnw`) — inclus, nu necesită instalare;
- **Docker + Docker Compose** — pentru PostgreSQL și stack-ul complet;
- **Node.js 20.19+** și **npm** — pentru frontend.

14.3 Scenarii de rulare

14.3.1 Doar backend pe profilul dev

```
docker compose up -d tastetest-app-db    # pornește PostgreSQL
./mvnw spring-boot:run                   # aplicația pe :8090
```

Cont admin implicit: `admin / admin@admin.com / admin`. Swagger UI: `http://localhost:8090/swagger-ui.html`.

14.3.2 Profilul test (H2, fără Docker)

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=test # pe :8091
```

Consola H2: `http://localhost:8091/h2-console` cu JDBC URL `jdbc:h2:mem:tastetest_test`, user `sa`, parolă goală.

14.3.3 Tot stack-ul cu Docker Compose

```
docker compose up --build
```

Serviciu	Acces
API Gateway (intrare publică)	http://localhost:8090
Frontend (shell) / MFE Admin	:4200 / :4201
Discovery (Eureka) / Config Server	:8761 / :8888
auth / user / restaurant	:8091 / :8092 / :8093
notification / agent (TasteBot)	:8082 / :8083
PostgreSQL / Redis / MongoDB	:5432 / :6379 / :27017
Prometheus / Grafana / Zipkin	:9090 / :3000 / :9411

14.3.4 Demo de load balancing și scalare

```
docker compose -f docker-compose.yml -f docker-compose.loadbalancing.yml \
  up --build --scale notification-service=2
```

Trimițând repetat cererea demo, câmpul `instanceId` din răspuns va alterna între instanțe — do-
vada că Spring Cloud LoadBalancer distribuie traficul:

```
curl -X POST http://localhost:8090/api/v1/integrations/notifications/demo \
  -H 'Content-Type: application/json' \
  -d '{"title":"Test","message":"Salut"}'
```

14.4 Healthcheck-uri și dependențe de pornire

Compose orchestrează ordinea de pornire prin healthcheck-uri: serviciile de business așteaptă ca Discovery, Config Server și PostgreSQL să fie `healthy` înainte să pornească; restaurant-service depinde în plus de Redis, iar notification-service de MongoDB. Astfel se evită erorile de „connection refused” la pornirea la rece.

14.5 Opreire

```
docker compose down          # oprește containerele
docker compose down -v       # oprește + șterge volumele (date persistate)
```

15. Troubleshooting și îmbunătățiri viitoare

15.1 Probleme frecvente

Simptom	Cauză / soluție
Database <code>"/Users/.../test"</code> not found în consola H2	JDBC URL greșit. Folosește <code>jdbc:h2:mem:taste-test_test</code> .
<code>Connection refused</code> la PostgreSQL pe dev	BD-ul nu rulează: <code>docker compose up -d taste-test-app-db</code> .
Admin role not found pe dev	Migrațiile Flyway nu au rulat. Verifică log-urile și că Flyway e activat în <code>application-dev.yml</code> .
Portul 8090/8091 e ocupat	Identifică procesul cu <code>lsof -i :8090</code> și oprește-l, sau schimbă portul în yml.
Agentul răspunde cu eroare 502	Modelul LLM nu e accesibil (cheie API lipsă sau LM Studio oprit). Verifică variabilele de mediu ale agent-service.
Gateway-ul dă 504 la chat	Inferența a depășit timeout-ul. Este deja ridicat la 150s; verifică disponibilitatea modelului.

15.2 Limitări cunoscute

- **Bază de date partajată** — serviciile partajează schema `project`, ceea ce le cuplează la nivel de schemă (vezi ADR din §6).
- **Consistență eventuală a rating-urilor** — o recenzie nouă poate dura până la 30s să se reflecte în `top-rated` (TTL Redis, §8.2).
- **Compensare best-effort** — dacă pasul de compensare al sagăi eșuează, sistemul rămâne inconsistent și necesită reconciliere manuală (§8.1).
- **Duplicarea în MFE** — modele/servicii sunt copiate între shell și mfe-admin; este prețul deliberat al independenței.
- **Secret JWT de dezvoltare** — valoarea implicită din config trebuie înlocuită în producție.

15.3 Direcții de îmbunătățire

- Introducerea unui **message broker** (Kafka/RabbitMQ) pentru invalidarea cross-service a cache-ului și pentru o saga prin coregrafie.
- **O bază de date per serviciu**, cu sincronizare prin evenimente, pentru decuplare completă.

- **Un job de reconciliere** pentru cazurile în care compensarea sagăi eșuează.
- **postMessage** între shell și MFE pentru logout instantaneu și integrare vizuală mai profundă.
- Finalizarea migrării din **backend-original** în microservicii dedicate (meniuri, categorii, feedback).
- Acoperire de teste extinsă (integrare, contract testing între servicii).

În rezumat

TasteTest este un sistem complet de microservicii care, dincolo de CRUD-ul de bază, demonstrează concret patternuri de sisteme distribuite: descoperire de servicii, configurare centralizată, gateway, securitate JWT stateless, o saga cu compensare, persistență poliglotă cu caching, micro-frontends, CI/CD și un agent AI integrat — fiecare cu o motivație arhitecturală explicită și cu compromisurile asumate documentat.